# El taller explicado fácil

Para entenderlo desde cero, sin código. Una idea por sección, cada una con un dibujo.

**Cómo leer esta guía:** cada sección tiene tres partes. Arriba, la idea **en pocas palabras**. En el medio, la explicación con un dibujo. Abajo, en amarillo, **lo que tienes que recordar**. Si solo lees los recuadros amarillos, ya tienes lo esencial.

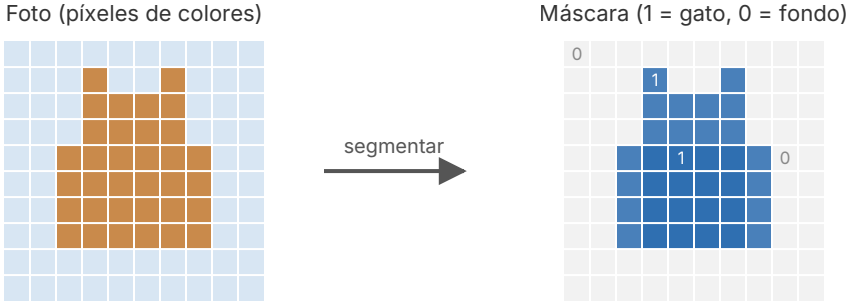

| # | Sección | La pregunta que responde |
|---|---|---|
| 1 | ¿Qué es segmentar? | ¿De qué trata todo esto? |
| 2 | La nota: IoU | ¿Cómo sabemos si un modelo lo hizo bien? |
| 3 | Los cuatro modelos | ¿Qué hace cada uno y en qué se diferencian? |
| 4 | SAM se confunde | ¿Por qué SAM necesita que le digan dónde mirar? |
| 5 | La competencia sin entrenar | ¿Quién ganó con las fotos de mascotas? |
| 6 | Peatones y U-Net++ | ¿Qué pasó al entrenar con nuestros datos? |
| 7 | El reto: YOLO + SAM | ¿Qué construimos? |
| 8 | Dónde perdimos | ¿En qué fotos salió peor? |
| 9 | Por qué ganamos | ¿De dónde salió la ganancia? |
| 10 | Palabras clave | Un mini diccionario |


## 1 ¿Qué es segmentar?

> **En pocas palabras:** segmentar es pintar, píxel por píxel, dónde está un objeto en una foto.

Una foto digital es una cuadrícula de puntitos de color llamados **píxeles**. Una foto de 500 × 333 tiene unos 166 000. Segmentar es decidir, para cada píxel, si pertenece al objeto (por ejemplo, el gato) o al fondo.

El resultado se llama **máscara**: una cuadrícula del mismo tamaño que la foto, con 1 donde está el gato y 0 donde no.



Hay varias formas de segmentar. Las dos que más se usan en el taller:

- **Semántica:** "estos píxeles son perro". Si hay dos perros, salen como una sola mancha.
- **De instancias:** "este es el perro 1 y este es el perro 2". Cada objeto tiene su propia máscara. Se usa para contar.

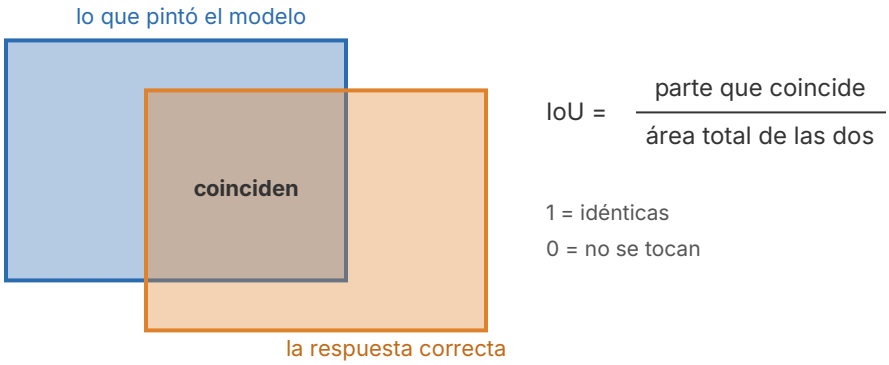
Para practicar usamos dos colecciones de fotos: **Oxford Pets** (fotos de un gato o un perro, cada una con su máscara correcta) y **PennFudanPed** (170 fotos de calle con 423 peatones marcados uno por uno).

**Lo que tienes que recordar:** segmentar = pintar dónde está el objeto. La máscara es la "respuesta" del modelo. Semántica junta todo; instancias separa cada objeto.

## 2 La nota: IoU

> **En pocas palabras:** el IoU es la nota de 0 a 1 que dice cuánto se parece la máscara del modelo a la respuesta correcta.

Imagina dos manchas: la que pintó el modelo y la correcta. El IoU compara la parte donde **coinciden** con toda el **área que cubren entre las dos**.




**Ejemplo con números:** el gato real tiene 100 píxeles. El modelo pintó 100 píxeles. Coinciden 80. Entre los dos cubren 100 + 100 − 80 = 120 píxeles. Entonces IoU = 80 ÷ 120 = **0.67**.

En el taller también aparece el **Dice**. Mide lo mismo, pero siempre da un poco más alto. En el ejemplo, Dice = 2 × 80 ÷ (100 + 100) = **0.80**.

**Un detalle:** en Pets, la respuesta correcta tiene una franja dudosa alrededor del animal (el "borde"), donde ni el que anotó sabía si era pelo o fondo. Esa franja **no cuenta** para la nota, para no castigar a nadie por algo ambiguo.

Para los peatones se usa otra nota, el **mask AP**. Es más exigente: revisa si cada peatón fue encontrado y bien recortado, con varios niveles de exigencia. También va de 0 a 1.

**Lo que tienes que recordar:** IoU = coincidencia ÷ total. Va de 0 a 1. Cuando decimos "le ganamos 0.9523 a 0.9419", hablamos del IoU promedio en 100 fotos.

## 3 Los cuatro modelos

> **En pocas palabras:** cada modelo segmenta de una forma distinta; ninguno es mejor en todo.

### U-Net / U-Net++

Mira la foto entera y pinta cada píxel.
Como un pintor que retoca el mismo cuadro varias veces.

Hay que entrenarlo con tus fotos.

### Mask R-CNN

Primero encierra cada objeto en una caja y luego recorta la silueta adentro.
Como encerrar y luego recortar.

Muy preciso, pero lento (~120 ms).

### YOLO-seg

Encuentra y recorta todo de una vez, mezclando 32 "plantillas" de forma.
Como armar figuras con un kit.

El más rápido (~19 ms).

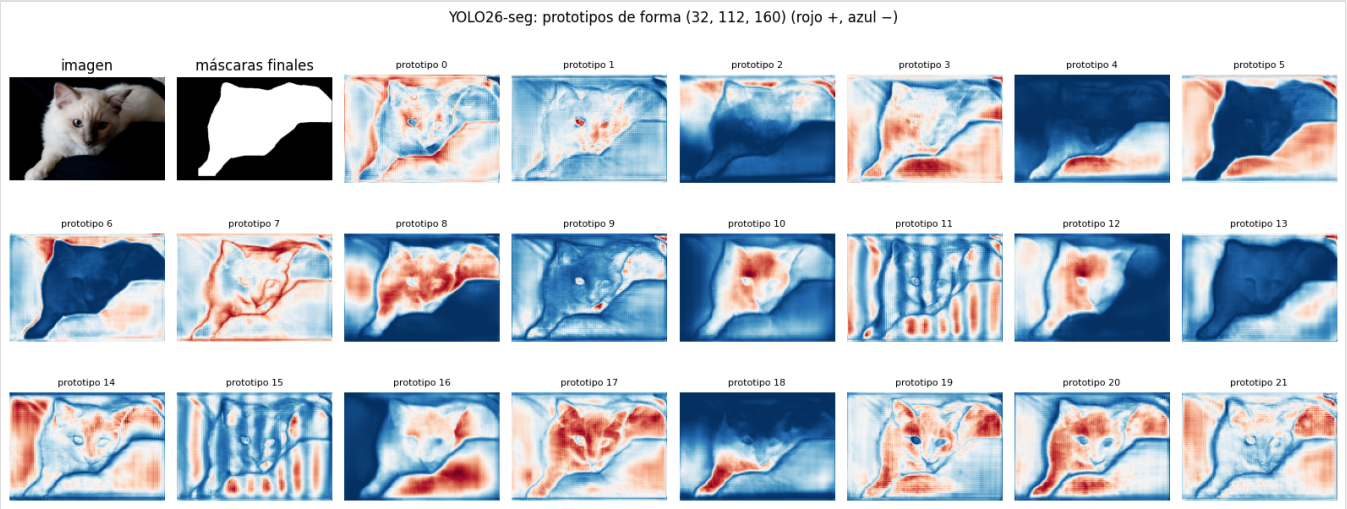
### SAM

Recorta lo que le señales con un punto o una caja. No sabe qué es nada.
Como un asistente: "recorta esto".

No hay que entrenarlo. Lento (~400 ms).

Dos de ellos ya vienen **preentrenados en COCO**, una colección enorme de fotos con 80 tipos de objetos (personas, gatos, perros, ovejas, osos…). Por eso Mask R-CNN y YOLO reconocen mascotas sin que les enseñemos nada. A eso se le dice **zero-shot**.

**Cómo funciona YOLO por dentro (la idea):** en vez de dibujar cada máscara desde cero, calcula 32 "plantillas" para toda la foto. Unas marcan el animal, otras el fondo, otras los bordes. Luego cada objeto mezcla esas plantillas en distinta proporción, como mezclar pinturas.




Las plantillas reales de YOLO sobre el gato blanco. Rojo = suma, azul = resta. La máscara final (blanca, arriba) es la mezcla de todas.

**El límite de Mask R-CNN:** recorta cada objeto en una cuadrícula de solo 28 × 28 cuadritos y luego la agranda. Por eso, aunque fuera perfecto, su nota máxima sería **0.978**: pierde detalles finos como orejas, colas y patas delgadas.

**Lo que tienes que recordar:** U-Net pinta todo; Mask R-CNN encierra y recorta; YOLO es el rápido; SAM recorta lo que le señales, pero no sabe qué es.

## 4 SAM se confunde si no le dicen bien dónde mirar

> **En pocas palabras:** SAM recorta muy bien, pero si la pista es ambigua elige mal, y además no se da cuenta.

Le pusimos a SAM un punto (la estrella roja) sobre el ojo del gato. Un punto así puede significar varias cosas: "el gato", "la cabeza" o "el ojo". Por eso SAM propone **tres máscaras** y le pone a cada una una nota de qué tan seguro está.




De izquierda a derecha: las tres propuestas con el punto, la propuesta con una caja, y la respuesta correcta.

| Propuesta | Qué tan seguro dijo estar SAM | Nota real (IoU) |
|---|---|---|
| 1. El gato completo | 0.90 | **0.98** |
| 2. Cabeza y pata | 0.80 | 0.72 |
| 3. Solo el ojo | **0.97** (la más segura) | 0.01 |
| Con una caja que cubre casi toda la foto | — | 0.80 (metió la manta) |

Fíjate en dos cosas:

- La propuesta de la que SAM estaba **más seguro** era la peor: solo el ojo. Su "seguridad" mide si la forma está bien dibujada, no si es lo que querías.
- Con una **caja enorme**, SAM no sabe qué es gato y qué es fondo, y mete la manta negra. **Esto es justo lo que pasa en el reto** cuando el animal llena la foto.

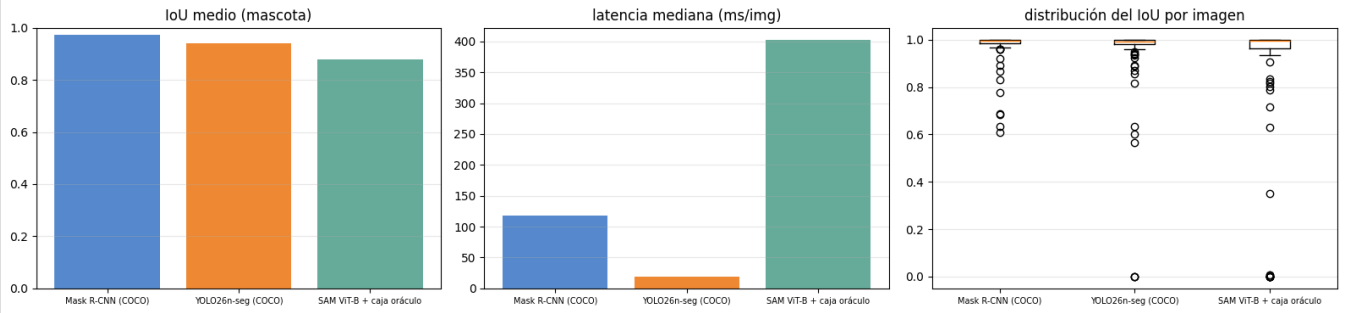
Cuando a SAM se le pide segmentar "todo" sin pistas, encuentra 13 objetos en la foto del perro (perro, cojín, sábana, nariz, orejas…), pero no sabe cuál es el perro.

**Lo que tienes que recordar:** SAM necesita que alguien le diga dónde mirar. Con un punto ambiguo o una caja que cubre toda la foto, se equivoca.

## 5 La competencia sin entrenar

> **En pocas palabras:** en 100 fotos de mascotas, Mask R-CNN fue el más preciso y YOLO el más rápido. YOLO perdió nota porque en 3 fotos no encontró a la mascota.

| Modelo | Nota (IoU) | Fotos sin encontrar nada | Tiempo por foto |
|---|---|---|---|
| Mask R-CNN | **0.972** | 0 | 119 ms |
| YOLO | 0.942 | **3** | **18 ms** |
| SAM con la caja correcta | 0.879 | 0 | 402 ms |




Izquierda: nota promedio. Centro: tiempo por foto. Derecha: nota de cada foto (cada círculo es una foto rara).

**Cómo leer la gráfica de la derecha:** la "cajita" de cada modelo muestra dónde quedó la mayoría de las fotos. Las tres están pegadas arriba, en 1.0: en casi todas las fotos los tres modelos son casi perfectos. Los **circulitos** son las fotos raras. YOLO tiene **3 circulitos abajo, en 0**: son las 3 fotos donde no encontró nada.

**La pista clave:** YOLO no recorta mal. Su promedio baja porque en 3 fotos **no encontró** a la mascota. Son dos problemas distintos: no encontrar y recortar mal. De ahí salió la idea del reto.

(SAM sacó 0.879 aunque le dimos la caja correcta porque en 2 fotos esa caja salió demasiado grande, por un error al construirla. Lo descubrimos en el ejercicio 3.)

> **Lo que tienes que recordar:** Mask R-CNN 0.972 (preciso), YOLO 0.942 (rápido). YOLO falla en *encontrar*, no en *recortar*.

## 6 Peatones y U-Net++

> **En pocas palabras:** entrenar con nuestros datos funcionó bien; YOLO quedó casi igual que Mask R-CNN pero mucho más rápido, y U-Net++ apenas le ganó a U-Net.

**Mask R-CNN con peatones.** Venía sabiendo 91 tipos de objetos. Le cambiamos la parte final para que solo distinga "peatón" o "fondo" y lo entrenamos 8 rondas (épocas). A esto se le llama **transfer learning**: aprovechar lo que ya sabía. Sacó un mask AP de **0.829**.




Dos peatones reales; Mask R-CNN encontró los dos y pintó a cada uno de un color (segmentación de instancias).

**YOLO contra Mask R-CNN con peatones:**

| | **Mask R-CNN** | **YOLO** |
|---|---|---|
| Nota (mask AP) | **0.829** | 0.810 |
| Tiempo por foto | 122 ms | **18 ms** (6.7× más rápido) |
| Tamaño | 45.9 millones de números | **3.1 millones** (15× más pequeño) |

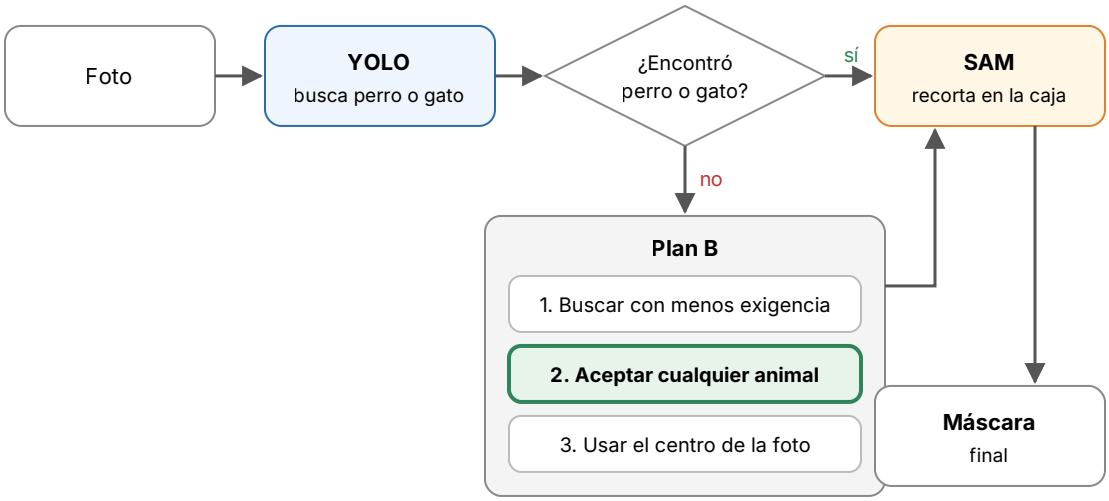
Para contar peatones en una cámara en vivo, YOLO es mejor elección: pierde muy poca nota y es muchísimo más rápido.

**U-Net contra U-Net++.** U-Net++ es una U-Net con más conexiones internas, que en teoría ayudan con los detalles finos. Resultado: **0.718 contra 0.709**. Gana por muy poco (más en los bordes), pero es un 44 % más lenta. Con una sola prueba, esa diferencia podría ser suerte.

**Lo que tienes que recordar:** con peatones, YOLO 0.810 vs Mask R-CNN 0.829, pero YOLO es 6.7× más rápido. U-Net++ apenas mejora a U-Net y es más lenta.

## 7 El reto: YOLO encuentra, SAM recorta

> **En pocas palabras:** armamos un equipo donde YOLO busca al perro o gato y SAM lo recorta. Si YOLO no encuentra nada, entra un plan B.




Se usó en 3 fotos; las 3 se resolvieron en el paso 2.

**Paso a paso:**

1. **YOLO busca** en la foto y devuelve cajas con un nombre (gato, perro, oveja…) y qué tan seguro está.
2. Nos quedamos solo con las cajas de **gato o perro** con seguridad de al menos 0.25 (25 %).
3. Si hay alguna, **SAM recorta** lo que hay dentro de cada caja. SAM "mira" la foto una sola vez y recorta todas las cajas juntas, para no gastar tiempo.
4. Si YOLO no encontró ningún gato o perro, entra el **plan B**: primero buscar con menos exigencia (5 %), luego aceptar la caja de **cualquier animal**, y si nada funciona, usar el centro de la foto.
5. Se juntan todos los recortes en una sola máscara.

**Reglas que cumplimos:** nunca usamos la respuesta correcta para ayudarnos, y no ajustamos números mirando las fotos una por una.

**Resultado: le ganamos a YOLO.** Nota **0.9523** contra **0.9419**. Las fotos sin encontrar nada bajaron de 3 a 0. Pero el tiempo subió de 19 a **469 ms** por foto, unas 25 veces más lento.

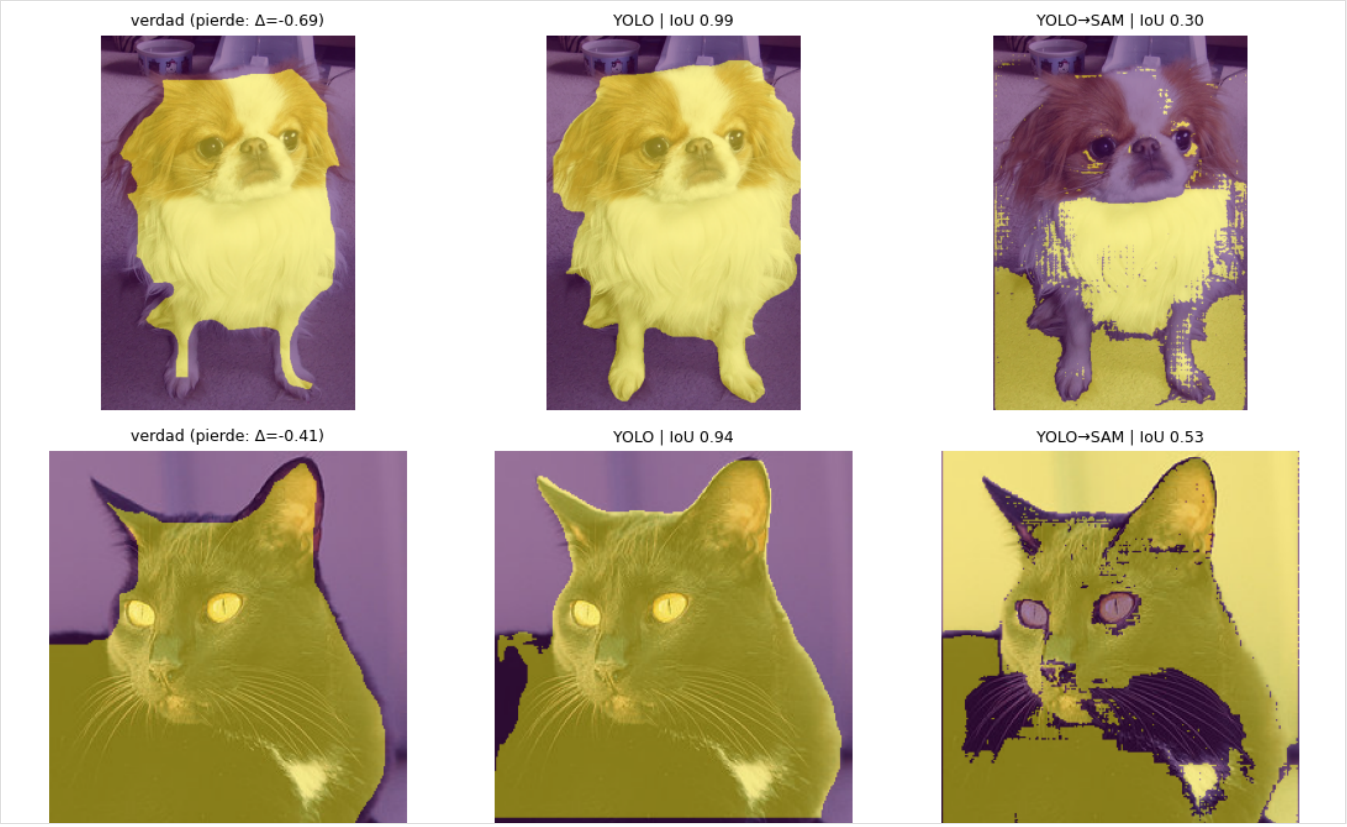
Al final también probamos con una foto propia, una pomerania, y la recortó bien, incluido el pelo esponjoso.




**Lo que tienes que recordar:** YOLO busca, SAM recorta, y si YOLO no encuentra nada, el plan B busca otra caja. Nota final 0.9523 contra 0.9419.

## 8 Dónde perdimos

> **En pocas palabras:** perdimos cuando el animal llenaba toda la foto: la caja era tan grande que SAM no supo qué era animal y qué era fondo.




Columnas: respuesta correcta · YOLO solo · nuestro equipo. Arriba, el perro de frente; abajo, el gato de frente.

- **Perro de frente:** YOLO lo tenía casi perfecto (0.99). Con SAM bajó a **0.30**: pintó el pecho y el suelo y dejó fuera la cara.
- **Gato de frente:** de 0.94 a **0.53**: SAM pintó la pared del fondo.

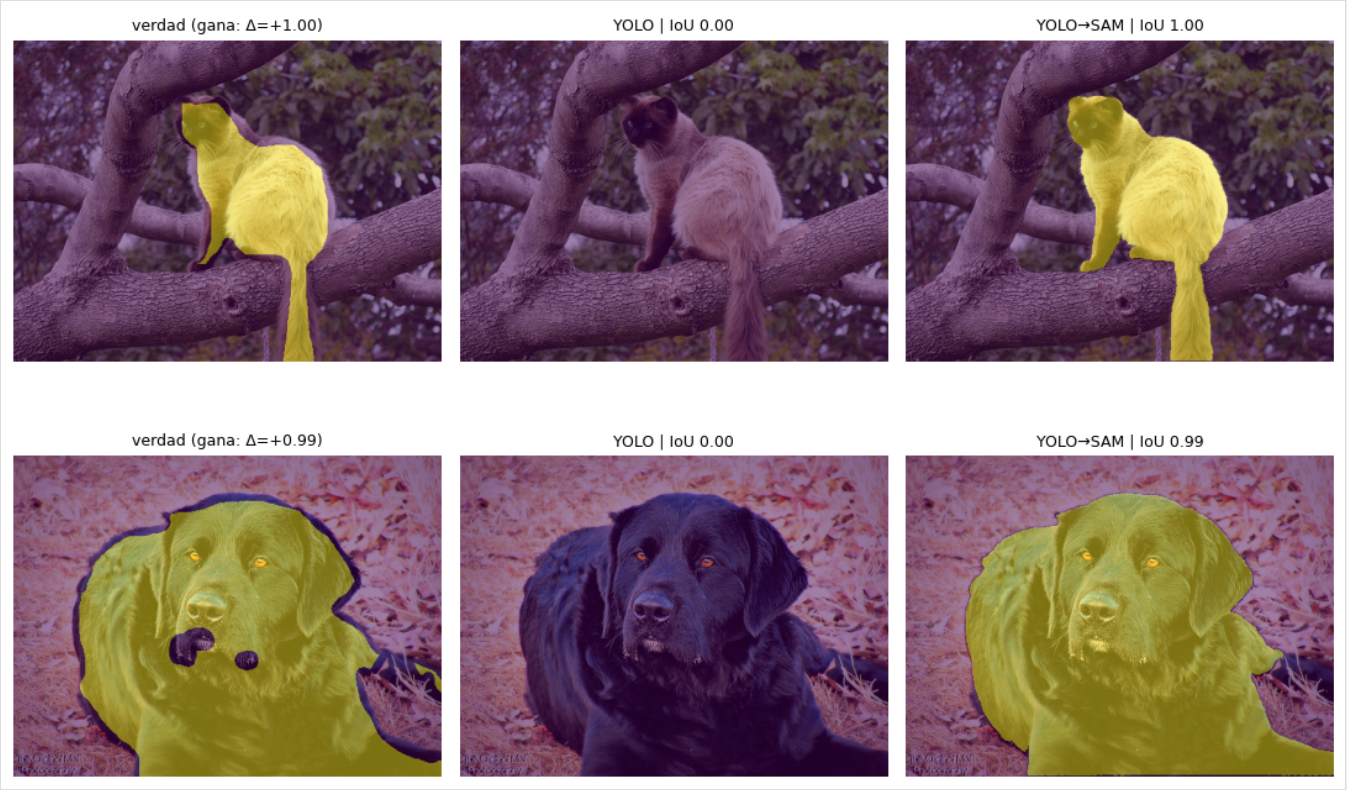
Es lo mismo que vimos en la sección 4: con una caja que cubre casi toda la foto, SAM se confunde. Y como YOLO ya era casi perfecto en esas fotos, cualquier error de SAM se convierte en pérdida.

**Las cuentas:** ganamos en 46 fotos, empatamos en 3 y perdimos en 51. Parece malo, pero la mayoría de las pérdidas son minúsculas: 31 de las 51 son de menos de 0.01, unos pocos píxeles del borde. Solo 12 fotos perdieron más de 0.05.

**Lo que tienes que recordar:** perdemos en primeros planos, porque la caja de YOLO cubre casi toda la foto y SAM confunde animal con fondo.

## 9 Por qué ganamos (lo más importante)

> **En pocas palabras:** no ganamos porque SAM recorte mejor. Ganamos porque el plan B rescató 3 fotos en las que YOLO vio al animal pero creyó que era una oveja o un oso.




Columnas: respuesta correcta · YOLO solo (no pintó nada) · nuestro equipo (casi perfecto).

En el gato siamés y el perro negro, YOLO sacó **0**: no encontró ningún gato ni perro. Pero sí había visto un animal: uno lo llamó **oveja** y los otros dos **oso**. El paso 2 del plan B tomó esa caja y se la pasó a SAM. Como a SAM no le importa el nombre, solo dónde está, los recortó casi perfecto: **0.989, 0.996 y 0.990**.




**Las cuentas finales:**

- El plan B sumó **+0.030** a la nota promedio.
- SAM, en las 97 fotos normales, **restó −0.020**.
- Resultado: **+0.010**. Sin el plan B, habríamos quedado en 0.9225, **por debajo de YOLO**, y no habríamos superado el reto.

**¿Vale la pena?** Para tiempo real, no: pasamos de unas 50 fotos por segundo a unas 2. Casi todo el tiempo extra es SAM "mirando" la foto (393 ms de los 469). La mejora lógica sería usar

SAM **solo cuando YOLO no encuentra nada**: con nuestros datos daría una nota cercana a 0.972 en unos 33 ms por foto.

**Lo que tienes que recordar:** ganamos gracias al plan B. YOLO vio a los animales pero los nombró mal (oveja, oso); a SAM no le importa el nombre, así que los recortó bien.

## 10 Palabras clave

| Palabra | Qué significa |
|---|---|
| Píxel | Cada puntito de color de una foto. |
| Máscara | La "pintura" del modelo: 1 donde está el objeto, 0 donde no. |
| Caja (bounding box) | Rectángulo que encierra un objeto. Se guarda con 4 números: esquina izquierda-arriba y derecha-abajo. |
| IoU | Nota de 0 a 1: parte que coincide ÷ área total de las dos máscaras. |
| Dice | Otra nota parecida al IoU; siempre da un poco más alto. |
| mask AP | Nota para instancias: revisa si cada objeto fue encontrado y bien recortado. |
| Trimap | La respuesta correcta de Pets: mascota, fondo y una franja de borde dudosa. |
| Prompt | La pista que se le da a SAM: un punto o una caja. |
| Encoder de SAM | La parte de SAM que "mira" la foto. Es la lenta (393 ms). |
| Decoder de SAM | La parte que recorta según la pista. Es rápida (14 ms). |
| Confianza (conf) | Qué tan seguro está YOLO de una detección, de 0 a 1. |
| COCO | Colección enorme de fotos con 80 tipos de objetos, con la que se entrenaron YOLO y Mask R-CNN. |
| Zero-shot | Usar un modelo en fotos nuevas sin reentrenarlo. |
| Transfer learning | Aprovechar un modelo ya entrenado y ajustarlo a tus datos. |
| Época | Una vuelta completa de entrenamiento por todas las fotos. |
| Latencia (ms) | Tiempo que tarda en procesar una foto. 1000 ms = 1 segundo. |
| GPU / VRAM | La tarjeta que hace los cálculos rápido y su memoria. Por falta de memoria el notebook borraba a SAM entre secciones. |
| Plan B (fallback) | Lo que hace nuestro equipo cuando YOLO no encuentra ningún gato o perro. |
| Huella SHA-256 | Una "firma" de nuestras 100 máscaras. Si cambia un píxel, cambia la firma. La nuestra empieza por 97ef26c2. |
| BGR / RGB | Dos órdenes de los colores. YOLO espera BGR; por eso la función to_bgr los voltea. |